Technology Explanation

# How The Technologies In The Project Work

A simple guide explaining Node.js, TypeScript, NestJS, Next.js, Prisma, PostgreSQL, Docker, JWT, REST APIs, and how they connect inside the College Support Multi-Agent Portal.

**For Presentation**

Explains the technologies in clear language for project defense or documentation.

**For This Project**

Shows exactly where each technology fits in your web app, API, database, and AI workflow.

Generated for the current project in `Transcendence-main`.

# 1. Big Picture

Your project is a full-stack web application. Full-stack means it has a frontend that users see in the browser, a backend that handles logic and security, and a database that stores the project data.

```
User browser
  -> Next.js frontend
  -> NestJS API running on Node.js
  -> Prisma ORM
  -> PostgreSQL database

Optional services
  -> Gemini AI for answers and embeddings
  -> SMTP mail server for email notifications
  -> Docker and Nginx for deployment
```

In simple words: the browser sends requests, the backend checks permissions and runs business logic, Prisma talks to the database, and PostgreSQL stores the information permanently.

# 2. Node.js

Node.js is a runtime that allows JavaScript and TypeScript code to run outside the browser. Normally JavaScript runs in a browser, but Node.js lets the backend server use JavaScript too.

## Why It Is Used

- It runs the NestJS backend API.
- It runs development tools like tests, build commands, Prisma commands, and package scripts.
- It lets the frontend and backend both use TypeScript/JavaScript.

## How It Works In Your Project

When you run `npm run start:dev` in the `api` folder, Node.js starts the NestJS server. That server listens for HTTP requests such as login, assistant messages, ticket creation, and admin actions.

# 3. npm

npm is the package manager used by Node.js projects. It installs libraries and runs scripts defined in `package.json`.

| Command | What It Does |
| --- | --- |
| `npm install` | Downloads project dependencies into `node_modules`. |
| `npm run dev` | Starts the frontend development server. |

| | |
|---|---|
| `npm run start:dev` | Starts the backend development server. |
| `npm run build` | Builds the project for production. |
| `npm test` | Runs backend tests. |

# 4. TypeScript

TypeScript is JavaScript with types. It helps developers catch mistakes before running the app. For example, it can warn when a function expects a number but receives a string.

## Why It Helps

- It makes code safer and easier to understand.
- It improves editor autocomplete and refactoring.
- It makes DTOs, API responses, roles, ticket statuses, and database types clearer.

Your frontend files use `.tsx` for React components, and your backend uses `.ts` for controllers, services, guards, DTOs, and modules.

# 5. Next.js

Next.js is the frontend framework in the `web` folder. It is built on React and provides routing, pages, build optimization, and production serving.

### How It Works

Next.js maps folders inside `web/src/app` to browser pages. For example, `web/src/app/login/page.tsx` becomes the login page, and `web/src/app/dashboard/page.tsx` becomes the dashboard page.

### In Your Project

- `/login` lets users sign in.
- `/register` lets students create accounts.
- `/dashboard` shows role-based tools for students, staff, and admins.
- `/dashboard/tickets/[id]` shows a specific ticket detail page.

# 6. React

React is the UI library used by Next.js. It lets the project build the interface using reusable components.

### How It Works

React components are functions that return UI. They can store state, respond to clicks, submit forms, show loading states, and update the page when data changes.

### In Your Project

- `DashboardClient.tsx` handles the main dashboard state and API calls.
- `TicketDetailClient.tsx` handles ticket messages, attachments, and ticket events.
- `AdminUsersPanel.tsx` lets admins assign departments and support areas.
- `AppHeader.tsx` displays the top navigation/header.

# 7. NestJS

NestJS is the backend framework in the `api` folder. It gives structure to the backend by organizing code into modules, controllers, services, guards, and DTOs.

| NestJS Part | Meaning | Example In Project |
|---|---|---|
| Module | Groups related backend features. | `TicketsModule`, `AuthModule`, `AssistantModule` |

| | | |
|---|---|---|
| Controller | Receives HTTP requests and returns responses. | `TicketsController`, `AssistantController` |
| Service | Contains business logic. | `TicketsService`, `KnowledgeService` |
| Guard | Checks whether a request is allowed. | `JwtAuthGuard`, `RolesGuard`, `ApiKeyGuard` |
| DTO | Defines and validates request data. | `CreateTicketDto`, `CreateFaqDto` |

# 8. REST API

A REST API is a way for the frontend and backend to communicate through HTTP endpoints. The frontend sends requests like `POST /api/auth/login`, and the backend returns JSON responses.

## Common HTTP Methods

| Method | Purpose | Example |
|---|---|---|
| `GET` | Read data. | `GET /api/tickets/my` |
| `POST` | Create data or perform an action. | `POST /api/assistant/message` |
| `PATCH` | Partially update data. | `PATCH /api/tickets/:id/status` |
| `PUT` | Update a resource. | `PUT /api/tickets/:id` |
| `DELETE` | Delete data. | `DELETE /api/tickets/:id/attachments/:attachmentId` |

# 9. PostgreSQL

PostgreSQL is the relational database used by the project. It stores data in tables with rows and columns. A relational database is good when data has clear relationships, such as users having tickets, tickets having events, and documents having chunks.

### How It Stores Your Data

| Table | What It Stores |
|---|---|
| `users` | Students, staff, and admins. |
| `tickets` | Support tickets created by students or the assistant. |
| `ticket_events` | History of actions on tickets. |
| `knowledge_documents` | Documents used by the knowledge agent. |
| `knowledge_chunks` | Text chunks and embeddings for retrieval. |
| `faq_entries` | Admin-created FAQ question-answer entries. |
| `notifications` | User notifications and read/unread state. |

PostgreSQL keeps this information even after the app restarts. With Docker Compose, the data is stored in a persistent Docker volume.

# 10. Prisma

Prisma is the ORM used by the backend. ORM means Object-Relational Mapper. It lets the TypeScript backend work with database records using TypeScript code instead of writing raw SQL for every operation.

### How Prisma Works

1. You define database models in `api/prisma/schema.prisma`.
2. Prisma reads the schema and generates a typed client.
3. The backend imports `PrismaService` and calls methods such as `prisma.user.findUnique()` or `prisma.ticket.create()`.
4. Prisma converts those method calls into SQL and sends them to PostgreSQL.
5. PostgreSQL returns data, and Prisma converts it back into TypeScript objects.

### Example Concept

```
Backend code:
  prisma.ticket.create({ data: { subject, description, studentId } })

Prisma:
  converts the TypeScript object into SQL

PostgreSQL:
  inserts a new row into the tickets table
```

Prisma is helpful because it reduces SQL mistakes, gives autocomplete, and keeps the database schema clear in one file.

## 11. pgvector And Embeddings

pgvector is a PostgreSQL extension for storing and comparing vector data. A vector is a list of numbers that represents meaning. In AI search, similar text usually has similar vectors.

In your project, knowledge chunks store embeddings as number arrays. When a student asks a question, the question is also embedded. The system compares the question vector with document vectors to find relevant knowledge.

```
Question: "What is the Student Affairs phone number?"
  -> converted to embedding numbers
  -> compared with knowledge chunk embeddings
  -> closest chunks are selected
  -> Gemini/local answer uses only selected context
```

## 12. JWT Authentication

JWT means JSON Web Token. It is used to prove that a user is logged in. After login, the backend returns an access token. The frontend stores it and sends it with later API requests.

### How It Works

```
User logs in
  -> backend verifies school ID and password
  -> backend signs a JWT
  -> frontend stores token in localStorage
  -> frontend sends Authorization: Bearer token
  -> JwtAuthGuard verifies token
  -> API knows the user role and permissions
```

In your project, the JWT contains the user ID, school ID, role, support area, and academic department. That helps the backend enforce permissions.

# 13. Guards And Permissions

Guards are backend checks that run before a request reaches the actual controller logic.

| Guard | Purpose |
| --- | --- |
| `JwtAuthGuard` | Requires the user to be logged in. |
| `RolesGuard` | Checks whether the user role is allowed to access the route. |
| `ApiKeyGuard` | Protects external public API routes using the `x-api-key` header. |

Services also perform deeper permission checks. For example, `TicketsService` checks that students only view their own tickets and staff only view tickets in their assigned support lane.

# 14. Docker

Docker packages applications and their dependencies into containers. A container runs the same way on different machines, which makes deployment easier.

## In Your Project

`docker-compose.yml` starts four services:

| Service | What It Runs |
| --- | --- |
| `db` | PostgreSQL database with pgvector. |
| `api` | NestJS backend server. |
| `web` | Next.js frontend app. |
| `reverse-proxy` | Nginx server that exposes the web app and API through one address. |

# 15. Nginx Reverse Proxy

Nginx is used as a reverse proxy. A reverse proxy receives browser requests and forwards them to the correct internal service.

```
Browser opens http://localhost
  -> Nginx receives request
  -> page request goes to web container
  -> /api request goes to api container
  -> browser sees one clean application address
```

This keeps deployment cleaner because users do not need to know separate frontend and backend ports.

# 16. Gemini AI

Gemini is the AI provider used by the project for answering with context and creating embeddings. It is optional during development because the project has fallback logic when `GEMINI_API_KEY` is missing.

## Two Jobs

- **Embedding:** Convert text into numbers that represent meaning.
- **Answering:** Generate an answer from the retrieved context.

The assistant is designed to avoid unsafe answers. If the trusted context is missing, it should say the context is insufficient and escalate to a support ticket.

# 17. How Everything Works Together

## Student Login Flow

```
Login page
  -> POST /api/auth/login
  -> AuthService checks user and Argon2 password hash
  -> JWT token returned
  -> frontend stores token
  -> dashboard loads user data
```

## Ask Assistant Flow

```
Dashboard assistant form
  -> POST /api/assistant/message
  -> Orchestrator classifies intent
  -> Knowledge agent searches Prisma/PostgreSQL
  -> Gemini/local fallback creates answer
  -> low confidence creates ticket suggestion
  -> trace is saved for admin review
```

## Create Ticket Flow

```
Student creates ticket
  -> TicketsController receives request
  -> TicketsService validates student and department
  -> Prisma creates ticket and event in PostgreSQL
  -> staff notifications are created
  -> dashboard refreshes ticket list
```

## Admin Knowledge Upload Flow

```
Admin uploads document
  -> KnowledgeController receives file/text
  -> KnowledgeService stores document
  -> text is split into chunks
  -> embeddings are generated
  -> chunks are saved for future question answering
```

# 18. Why These Technologies Are Good For This Project

| Technology | Why It Fits |
| --- | --- |
| Node.js | Runs the TypeScript backend and tooling. |
| TypeScript | Makes both frontend and backend safer and easier to maintain. |

| | |
|---|---|
| Next.js | Provides a modern web interface with routing and production build support. |
| NestJS | Gives the backend a clean enterprise-style structure. |
| Prisma | Simplifies database access and keeps schema/models strongly typed. |
| PostgreSQL | Stores relational data reliably and supports advanced extensions. |
| Docker | Makes running the full stack easier and more consistent. |
| JWT | Supports stateless login sessions for the API. |
| Gemini | Adds AI answers and semantic knowledge retrieval. |

## 19. Simple Definitions

| Term | Simple Meaning |
|---|---|
| Frontend | The part users see and click in the browser. |
| Backend | The server code that handles logic, security, and database operations. |
| Database | The place where data is stored permanently. |
| API | A set of endpoints that let frontend and backend communicate. |
| ORM | A tool that lets code work with database tables as objects. |
| DTO | A class that describes and validates request data. |
| JWT | A signed token proving the user is logged in. |
| Embedding | A list of numbers representing the meaning of text. |
| RAG | Retrieval-Augmented Generation: search trusted documents first, then generate an answer from them. |

## 20. Summary

The project works because each technology has a clear job. Next.js and React create the user interface. Node.js runs the backend. NestJS organizes backend logic. Prisma connects the backend to PostgreSQL. PostgreSQL stores the data. JWT and guards protect the system. Docker and Nginx make deployment easier. Gemini adds AI features for knowledge answering.

Together, these technologies create a complete support system where users can log in, ask questions, create tickets, manage queues, upload knowledge, and receive AI-assisted answers from trusted project data.